

Data Pipelines with Apache Airflow 3.2

By, Bhavani Ravi

thelearning.dev

Setup

Data Pipelines with Apache Airflow 3.2

Clone or fork the repo, follow the Readme.md instructions

<https://github.com/thelearningdev/pyconus-2026-apache-airflow-tutorial>

or

<https://bit.ly/pycon2026-airflow-tutorial>

Apache Airflow

Data Engineering

In today's Session

1. Introduction to Apache Airflow
2. Data Ingestion & Idempotency
3. Backfill, incremental load and scheduling
4. Data validation and quarantine
5. Build Report

The Bookshop

WHAT THE BUSINESS WANTS

WHAT THE TEAM ACTUALLY HAS

The Bookshop

WHAT THE BUSINESS WANTS

- Know which genres sell best each day
- Bad sales records flagged, not silently loaded
- Daily report ready without manual intervention

WHAT THE TEAM ACTUALLY HAS

The Bookshop

WHAT THE BUSINESS WANTS

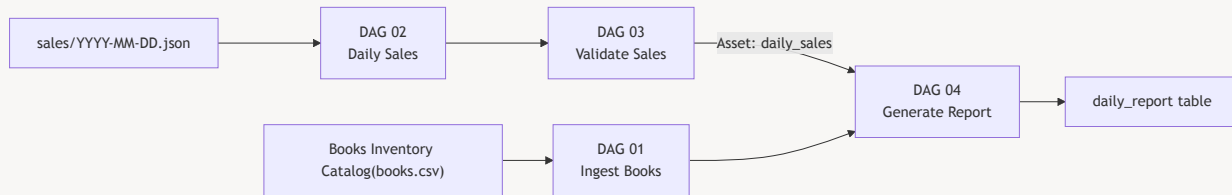
- Know which genres sell best each day
- Bad sales records flagged, not silently loaded
- Daily report ready without manual intervention

WHAT THE TEAM ACTUALLY HAS

- A 25-row books catalog CSV with a few bad rows
- Daily JSON sales files that contain invalid records
- Scripts that are not scheduled and not observable

The Final Pipeline

At the end of this workshop we will have...



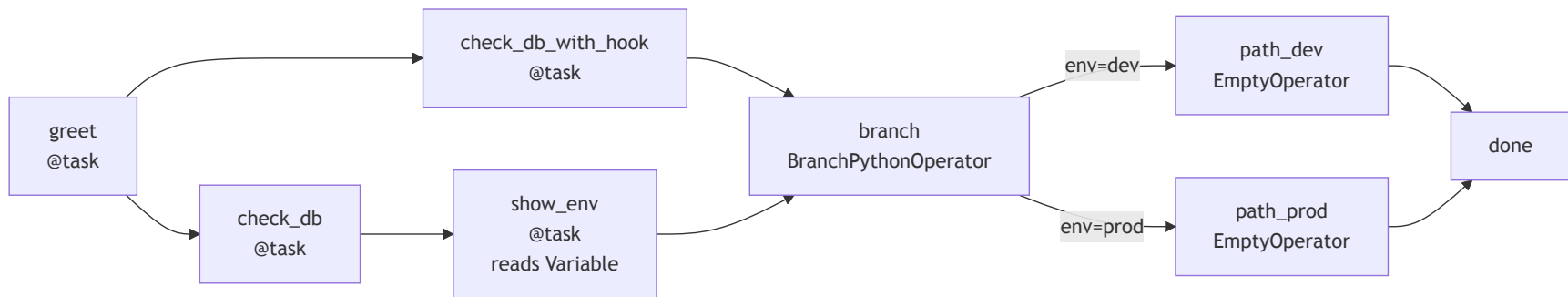
Apache Airflow 101

Hello World

Core Airflow Concepts in One DAG

Before the real pipeline, build one DAG that exercises every concept you will rely on.

Airflow 101 Dag



Exercise 0

Run the Hello World DAG

Trigger it, Some tasks will fail, check why

```
dags/00_hello_world.py
```

The DAG

```
@dag(
    dag_id="00_hello_world",
    start_date=datetime(2026, 1, 1),
    schedule=None,          # manual trigger only
    catchup=False,
    tags=["workshop", "hello-world"],
)
def hello_world():
    ...

hello_world()              # registers the DAG with Airflow
```

The DAG

```
@dag(
    dag_id="00_hello_world",
    start_date=datetime(2026, 1, 1),
    schedule=None,          # manual trigger only
    catchup=False,
    tags=["workshop", "hello-world"],
)
def hello_world():
    ...

hello_world()                # registers the DAG with Airflow
```

- A DAG is a Python function decorated with `@dag`

The DAG

```
@dag(
    dag_id="00_hello_world",
    start_date=datetime(2026, 1, 1),
    schedule=None,          # manual trigger only
    catchup=False,
    tags=["workshop", "hello-world"],
)
def hello_world():
    ...

hello_world()                # registers the DAG with Airflow
```

- A DAG is a Python function decorated with `@dag`
- `schedule=None` means the DAG only runs when you click **Trigger**

DAG

The DAG

```
@dag(
    dag_id="00_hello_world",
    start_date=datetime(2026, 1, 1),
    schedule=None,          # manual trigger only
    catchup=False,
    tags=["workshop", "hello-world"],
)
def hello_world():
    ...

hello_world()                # registers the DAG with Airflow
```

- A DAG is a Python function decorated with `@dag`
- `schedule=None` means the DAG only runs when you click **Trigger DAG**
- `catchup=False` prevents Airflow from backfilling historical runs on first load

The DAG

```
@dag(
    dag_id="00_hello_world",
    start_date=datetime(2026, 1, 1),
    schedule=None,          # manual trigger only
    catchup=False,
    tags=["workshop", "hello-world"],
)
def hello_world():
    ...

hello_world()                # registers the DAG with Airflow
```

- A DAG is a Python function decorated with `@dag`
- `schedule=None` means the DAG only runs when you click **Trigger DAG**
- `catchup=False` prevents Airflow from backfilling historical runs on first load

Tasks and Operators

Tasks and Operators

`@task` — Python shorthand

```
@task
def say_hello() → str:
    message = "Hello, Airflow 3!"
    print(message)
    return message

hello_task = say_hello()
```

Tasks and Operators

@task — Python shorthand

```
@task
def say_hello() → str:
    message = "Hello, Airflow 3!"
    print(message)
    return message

hello_task = say_hello()
```

Equivalent PythonOperator

```
def say_hello() → str:
    message = "Hello, Airflow 3!"
    print(message)
    return message

hello_task = PythonOperator(
    task_id="check_date",
    python_callable=say_hello()
)
```

Tasks and Operators

@task — Python shorthand

```
@task
def say_hello() → str:
    message = "Hello, Airflow 3!"
    print(message)
    return message

hello_task = say_hello()
```

Equivalent PythonOperator

```
def say_hello() → str:
    message = "Hello, Airflow 3!"
    print(message)
    return message

hello_task = PythonOperator(
    task_id="check_date",
    python_callable=say_hello()
)
```

Other SDK Alternatives

- @task.bash
- @task.branch
- @task.virtualenv

Task Chaining

```
greet_task = greet()
hook_task = check_db_with_hook()

greet_task >> hook_task >> branch
greet_task >> check_db() >> show_env() >> branch
branch >> [path_dev, path_prod]
[path_dev, path_prod] >> done
```

Task Chaining

```
greet_task = greet()
hook_task = check_db_with_hook()

greet_task >> hook_task >> branch
greet_task >> check_db() >> show_env() >> branch
branch >> [path_dev, path_prod]
[path_dev, path_prod] >> done
```

- `>>` declares a dependency: left must complete before right starts

Task Chaining

```
greet_task = greet()
hook_task = check_db_with_hook()

greet_task >> hook_task >> branch
greet_task >> check_db() >> show_env() >> branch
branch >> [path_dev, path_prod]
[path_dev, path_prod] >> done
```

- `>>` declares a dependency: left must complete before right starts
- Putting tasks in a list `[a, b] >> c` means both `a` and `b` must finish

Task Chaining

```
greet_task = greet()
hook_task = check_db_with_hook()

greet_task >> hook_task >> branch
greet_task >> check_db() >> show_env() >> branch
branch >> [path_dev, path_prod]
[path_dev, path_prod] >> done
```

- `>>` declares a dependency: left must complete before right starts
- Putting tasks in a list `[a, b] >> c` means both `a` and `b` must finish
- Airflow resolves the full dependency graph before scheduling any task

BranchPythonOperator

```
def _pick_branch() → str:
    env = Variable.get("bookshop_env", default_var="dev")
    return "path_prod" if env == "prod" else "path_dev"

branch = BranchPythonOperator(
    task_id="branch",
    python_callable=_pick_branch,
)

branch >> [path_dev, path_prod]
```

BranchPythonOperator

```
def _pick_branch() → str:
    env = Variable.get("bookshop_env", default_var="dev")
    return "path_prod" if env == "prod" else "path_dev"

branch = BranchPythonOperator(
    task_id="branch",
    python_callable=_pick_branch,
)

branch >> [path_dev, path_prod]
```

- Returns the `task_id` (or a list) of the branch to follow

BranchPythonOperator

```
def _pick_branch() → str:
    env = Variable.get("bookshop_env", default_var="dev")
    return "path_prod" if env == "prod" else "path_dev"

branch = BranchPythonOperator(
    task_id="branch",
    python_callable=_pick_branch,
)

branch >> [path_dev, path_prod]
```

- Returns the `task_id` (or a list) of the branch to follow
- Tasks on the unchosen path are **skipped**, not failed

BranchPythonOperator

```
def _pick_branch() → str:
    env = Variable.get("bookshop_env", default_var="dev")
    return "path_prod" if env == "prod" else "path_dev"

branch = BranchPythonOperator(
    task_id="branch",
    python_callable=_pick_branch,
)

branch >> [path_dev, path_prod]
```

- Returns the `task_id` (or a list) of the branch to follow
- Tasks on the unchosen path are **skipped**, not failed
- Real use cases: environment checks, feature flags, data-driven routing

Trigger Rules

```
join = EmptyOperator(  
    task_id="join",  
    trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,  
)
```

Default: ALL_SUCCESS

NONE_FAILED_MIN_ONE_SUCCESS

Trigger Rules

```
join = EmptyOperator(  
    task_id="join",  
    trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,  
)
```

Default: ALL_SUCCESS

NONE_FAILED_MIN_ONE_SUCCESS

- Every upstream task must succeed

Trigger Rules

```
join = EmptyOperator(  
    task_id="join",  
    trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,  
)
```

Default: ALL_SUCCESS

NONE_FAILED_MIN_ONE_SUCCESS

- Every upstream task must succeed
- A skipped branch **blocks** the downstream task

Trigger Rules

```
join = EmptyOperator(  
    task_id="join",  
    trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,  
)
```

Default: ALL_SUCCESS

NONE_FAILED_MIN_ONE_SUCCESS

- Every upstream task must succeed
- A skipped branch **blocks** the downstream task
- Works fine when there is no branching

Trigger Rules

```
join = EmptyOperator(  
    task_id="join",  
    trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,  
)
```

Default: ALL_SUCCESS

- Every upstream task must succeed
- A skipped branch **blocks** the downstream task
- Works fine when there is no branching

NONE_FAILED_MIN_ONE_SUCCESS

- No upstream task failed

Trigger Rules

```
join = EmptyOperator(  
    task_id="join",  
    trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,  
)
```

Default: ALL_SUCCESS

- Every upstream task must succeed
- A skipped branch **blocks** the downstream task
- Works fine when there is no branching

NONE_FAILED_MIN_ONE_SUCCESS

- No upstream task failed
- At least one upstream task succeeded

Trigger Rules

```
join = EmptyOperator(  
    task_id="join",  
    trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,  
)
```

Default: ALL_SUCCESS

- Every upstream task must succeed
- A skipped branch **blocks** the downstream task
- Works fine when there is no branching

NONE_FAILED_MIN_ONE_SUCCESS

- No upstream task failed
- At least one upstream task succeeded
- Skipped tasks are acceptable — the join proceeds

Trigger Rules

```
join = EmptyOperator(  
    task_id="join",  
    trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,  
)
```

Default: ALL_SUCCESS

- Every upstream task must succeed
- A skipped branch **blocks** the downstream task
- Works fine when there is no branching

NONE_FAILED_MIN_ONE_SUCCESS

- No upstream task failed
- At least one upstream task succeeded
- Skipped tasks are acceptable — the join proceeds

Connections

1. A named, encrypted credential stored in Airflow. Your DAG code refers to it by ID — the actual host, port, and password live in the UI, not in the code
2. To create it, Go to Airflow UI → Admin → Connections → + Add. Set Conn ID to `bookshop_postgres`, Conn Type to `Postgres`

Connections

1. A named, encrypted credential stored in Airflow. Your DAG code refers to it by ID — the actual host, port, and password live in the UI, not in the code
2. To create it, Go to Airflow UI → Admin → Connections → + Add. Set Conn ID to `bookshop_postgres` , Conn Type to `Postgres`

Used when: The same DAG file works in dev and prod because the connection ID stays the same — only the credentials change per environment.

Connections

1. A named, encrypted credential stored in Airflow. Your DAG code refers to it by ID — the actual host, port, and password live in the UI, not in the code
2. To create it, Go to Airflow UI → Admin → Connections → + Add. Set Conn ID to `bookshop_postgres` , Conn Type to `Postgres`

Used when: The same DAG file works in dev and prod because the connection ID stays the same — only the credentials change per environment.

Providers

- Providers are python packages that follow airflow specs
- This enables airflow to connect, run task, pull data from an external system
- Common providers include `standard`, `postgres`, `aws`, `google`

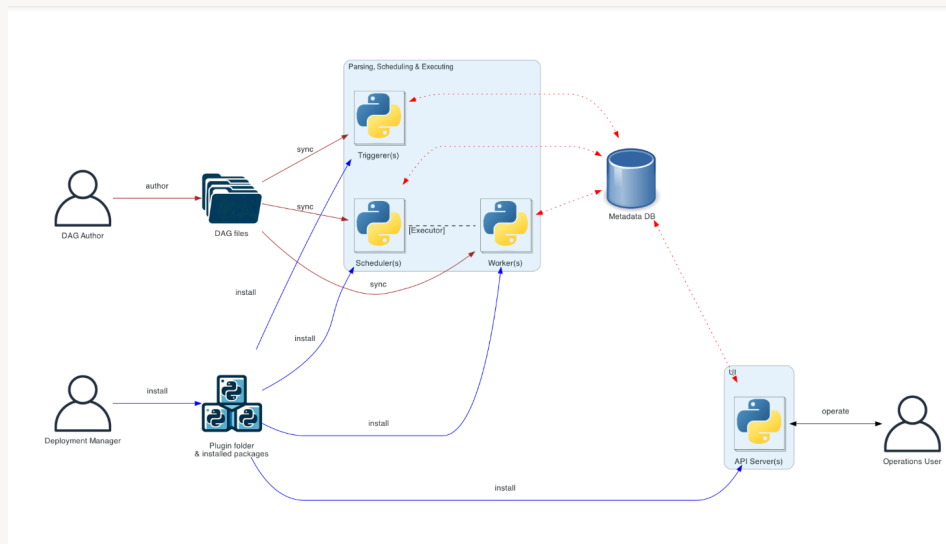
Mini-Exercise

Explore Provider Packages

<https://airflow.apache.org/docs/#providers-packages>

Airflow Architecture

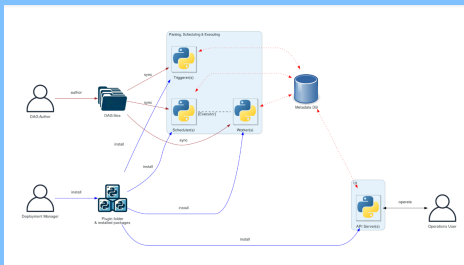
- Scheduler
- API Server
- Executor/Worker
- Triggerers
- Dag Processor
- Metadata DB



The diagram illustrates the Python Machine Learning Architecture. It shows the flow from a User/Manager to Model Training and Deployment. The User/Manager interacts with the Model Training process, which involves a Jupyter Notebook and a Python script. The Model Training process also interacts with a Database and a Deployment process. The Deployment process involves a Python script and a Deployment Manager.

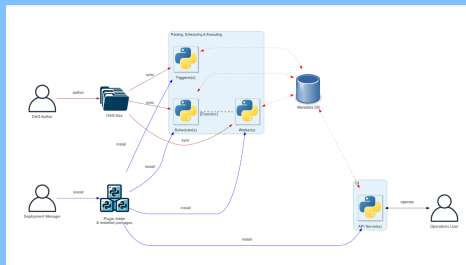
```

graph LR
    User((User/Manager)) -- "upload" --> Jupyter[Jupyter Notebook]
    Jupyter -- "train" --> Python[Python script]
    Python -- "train" --> Model[Model]
    Model -- "train" --> DB[(Database)]
    DB -- "train" --> Python
    Python -- "train" --> Deployment[Deployment]
    Deployment -- "train" --> Manager((Deployment Manager))
  
```

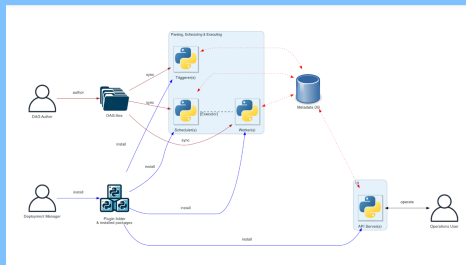


Airflow Architecture

- **Scheduler** — continuously parses DAG files and queues tasks that are ready to run

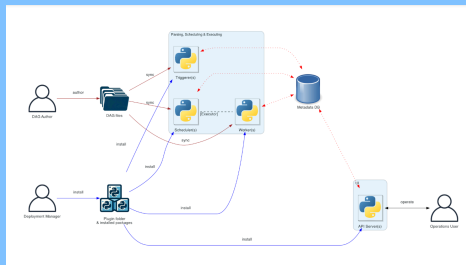


Airflow Architecture



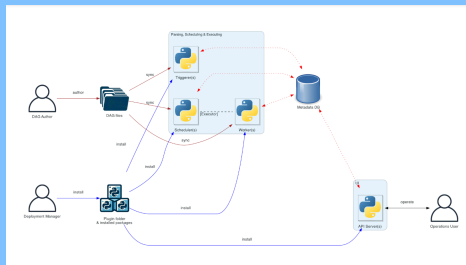
- **Scheduler** — continuously parses DAG files and queues tasks that are ready to run
- **API Server** — serves the UI and the REST API; reads state from the metadata DB

Airflow Architecture



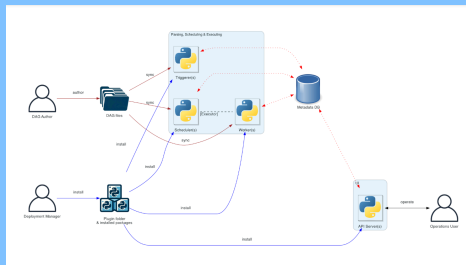
- **Scheduler** — continuously parses DAG files and queues tasks that are ready to run
- **API Server** — serves the UI and the REST API; reads state from the metadata DB
- **Executor** — decides where tasks run (same process, separate process, or remote worker)

Airflow Architecture



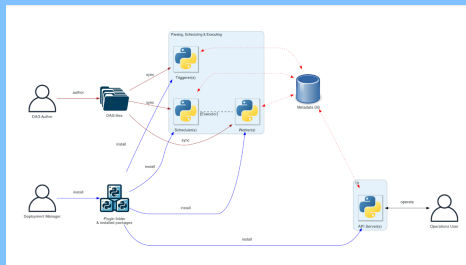
- **Scheduler** — continuously parses DAG files and queues tasks that are ready to run
- **API Server** — serves the UI and the REST API; reads state from the metadata DB
- **Executor** — decides where tasks run (same process, separate process, or remote worker)
- **Triggerer** — like worker, but for deferred tasks so it doesn't block other work

Airflow Architecture



- **Scheduler** — continuously parses DAG files and queues tasks that are ready to run
- **API Server** — serves the UI and the REST API; reads state from the metadata DB
- **Executor** — decides where tasks run (same process, separate process, or remote worker)
- **Triggerer** — like worker, but for deferred tasks so it doesn't block other work
- **Dag processor** — serializes DAGs and makes them available to other components

Airflow Architecture



- **Scheduler** — continuously parses DAG files and queues tasks that are ready to run
- **API Server** — serves the UI and the REST API; reads state from the metadata DB
- **Executor** — decides where tasks run (same process, separate process, or remote worker)
- **Triggerer** — like worker, but for deferred tasks so it doesn't block other work
- **Dag processor** — serializes DAGs and makes them available to other components
- **Metadata DB** — the source of truth for all run history, connections, and variables

Executors

*This workshop uses **SequentialExecutor** — one task at a time, no infrastructure setup.*

Executors

*This workshop uses **SequentialExecutor** — one task at a time, no infrastructure setup.*

- **SequentialExecutor** — one task at a time; default for `airflow` `standalone`

Executors

*This workshop uses **SequentialExecutor** — one task at a time, no infrastructure setup.*

- **SequentialExecutor** — one task at a time; default for `airflow standalone`
- **LocalExecutor** — parallel tasks on a single machine; good for team development

Executors

*This workshop uses **SequentialExecutor** — one task at a time, no infrastructure setup.*

- **SequentialExecutor** — one task at a time; default for `airflow` `standalone`
- **LocalExecutor** — parallel tasks on a single machine; good for team development
- **CeleryExecutor / KubernetesExecutor** — distributed workers for production scale

Executors

*This workshop uses **SequentialExecutor** — one task at a time, no infrastructure setup.*

- **SequentialExecutor** — one task at a time; default for `airflow standalone`
- **LocalExecutor** — parallel tasks on a single machine; good for team development
- **CeleryExecutor / KubernetesExecutor** — distributed workers for production scale

Changing the executor will not change how to Dag Runs.

Airflow 101: Reflections



Airflow pipeline concepts - Dag, Task, Variables, Connections...



DAG Code Concepts - Chaining, Branching, Trigger Rules



Airflow Infrastructure



Airflow is still an orchestration engine, not a transformation engine

Data Engineering

Data Engineering

Data Engineering

MOVING DATA

Getting raw data from sources (files, APIs, databases) into a place where it can be used.

Data Engineering

MOVING DATA

Getting raw data from sources (files, APIs, databases) into a place where it can be used.

TRANSFORMING DATA

Cleaning, validating, joining, and reshaping data so it is fit for a specific question.

Data Engineering

MOVING DATA

Getting raw data from sources (files, APIs, databases) into a place where it can be used.

TRANSFORMING DATA

Cleaning, validating, joining, and reshaping data so it is fit for a specific question.

SERVING DATA

Making the result reliably available — to a dashboard, an API, an ML model, or another team.

Data Engineering

MOVING DATA

Getting raw data from sources (files, APIs, databases) into a place where it can be used.

TRANSFORMING DATA

Cleaning, validating, joining, and reshaping data so it is fit for a specific question.

SERVING DATA

Making the result reliably available — to a dashboard, an API, an ML model, or another team.

DATA ENGINEERING IS THE PLUMBING

Analysts and scientists depend on it being invisible when it works and loud when it breaks.

Orchestration

SCHEDULING

Run this pipeline daily at 06:00, or whenever the upstream data arrives.

Orchestration

SCHEDULING

Run this pipeline daily at 06:00, or whenever the upstream data arrives.

DEPENDENCY MANAGEMENT

Do not start the enrichment step until the ingest step has finished successfully.

Orchestration

SCHEDULING

Run this pipeline daily at 06:00, or whenever the upstream data arrives.

DEPENDENCY MANAGEMENT

Do not start the enrichment step until the ingest step has finished successfully.

RETRY AND ALERTING

If a step fails, retry it up to three times, then page someone.

Orchestration

SCHEDULING

Run this pipeline daily at 06:00, or whenever the upstream data arrives.

DEPENDENCY MANAGEMENT

Do not start the enrichment step until the ingest step has finished successfully.

RETRY AND ALERTING

If a step fails, retry it up to three times, then page someone.

OBSERVABILITY

Show exactly what ran, when, how long it took, and what output it produced.

Orchestration

SCHEDULING

Run this pipeline daily at 06:00, or whenever the upstream data arrives.

DEPENDENCY MANAGEMENT

Do not start the enrichment step until the ingest step has finished successfully.

RETRY AND ALERTING

If a step fails, retry it up to three times, then page someone.

OBSERVABILITY

Show exactly what ran, when, how long it took, and what output it produced.

AIRFLOW'S ROLE

Airflow is the orchestrator. Your Python code is the work. Airflow manages everything around it.

DagRun

- Every time we trigger a pipeline, an instance(object) of the dag is created
- This instance has the information of all tasks that needs to run in that pipeline
- It has a state success/failure
- You can clear a DagRun to run again in the same instance

Task Instances

- Similar to DagRun, TaskInstances are created for per task per dag on trigger
- TaskInstances has states like `Success, Failure, Skipped, Running`
- You can clear a specific task to rerun again

Mini-Exercise

Explore DagRun and Task Instance in UI

Module 1

Ingest Data

Catalog Data

(books.csv)

ISBN	Title	Author	Genre	Price
9780743273565	The Great Gatsby	F. Scott Fitzgerald	Fiction	12.99
9780061096525	To Kill a Mockingbird	Harper Lee	Fiction	13.99
9780743477550	1984	George Orwell	Fiction	11.99

Connections (Recap)

1. A named, encrypted credential stored in Airflow.
2. Your DAG code refers to it by ID
3. The actual credentials host, port, and password live in the Airflow DB, not in your code.
4. You can also use external secret managers

We can use the same credential across multiple dags

Hooks with Connection

```
from airflow.providers.postgres.hooks.postgres import PostgresHook

hook = PostgresHook(postgres_conn_id="bookshop_postgres")

# Run a query, get results as list of tuples
rows = hook.get_records("SELECT isbn, title FROM books")

# Bulk insert rows
hook.insert_rows(
    table="books",
    rows=[("978 ... ", "Dune", "Frank Herbert", "Sci-Fi", 16.99)],
    target_fields=["isbn", "title", "author", "genre", "price"],
)
```

Hooks with Connection

```
from airflow.providers.postgres.hooks.postgres import PostgresHook

hook = PostgresHook(postgres_conn_id="bookshop_postgres")

# Run a query, get results as list of tuples
rows = hook.get_records("SELECT isbn, title FROM books")

# Bulk insert rows
hook.insert_rows(
    table="books",
    rows=[("978 ... ", "Dune", "Frank Herbert", "Sci-Fi", 16.99)],
    target_fields=["isbn", "title", "author", "genre", "price"],
)
```

- `get_records(sql)` — returns a list of tuples

Hooks with Connection

```
from airflow.providers.postgres.hooks.postgres import PostgresHook

hook = PostgresHook(postgres_conn_id="bookshop_postgres")

# Run a query, get results as list of tuples
rows = hook.get_records("SELECT isbn, title FROM books")

# Bulk insert rows
hook.insert_rows(
    table="books",
    rows=[("978 ... ", "Dune", "Frank Herbert", "Sci-Fi", 16.99)],
    target_fields=["isbn", "title", "author", "genre", "price"],
)
```

- `get_records(sql)` — returns a list of tuples
- `get_df(sql)` — returns a pandas DataFrame

Hooks with Connection

```
from airflow.providers.postgres.hooks.postgres import PostgresHook

hook = PostgresHook(postgres_conn_id="bookshop_postgres")

# Run a query, get results as list of tuples
rows = hook.get_records("SELECT isbn, title FROM books")

# Bulk insert rows
hook.insert_rows(
    table="books",
    rows=[("978 ... ", "Dune", "Frank Herbert", "Sci-Fi", 16.99)],
    target_fields=["isbn", "title", "author", "genre", "price"],
)
```

- `get_records(sql)` — returns a list of tuples
- `get_df(sql)` — returns a pandas DataFrame
- `insert_rows( ... )` — bulk insert with optional upsert support

Hooks with Connection

```
from airflow.providers.postgres.hooks.postgres import PostgresHook

hook = PostgresHook(postgres_conn_id="bookshop_postgres")

# Run a query, get results as list of tuples
rows = hook.get_records("SELECT isbn, title FROM books")

# Bulk insert rows
hook.insert_rows(
    table="books",
    rows=[("978 ... ", "Dune", "Frank Herbert", "Sci-Fi", 16.99)],
    target_fields=["isbn", "title", "author", "genre", "price"],
)
```

- `get_records(sql)` — returns a list of tuples
- `get_df(sql)` — returns a pandas DataFrame
- `insert_rows( ... )` — bulk insert with optional upsert support
- No need to manage connections manually — the hook handles open/close

Exercise 1

Complete `ingest_books` Dag

Idempotency

- Idempotency ensures that running a data pipeline multiple times with the same input produces the exact same output, preventing duplicates and data corruption
- Consistent Results
- Duplicate Prevention
- Fault tolerance

Full-Load

- The act of replacing the old data with full-set of new data is called Full-load.
- Also known as `bulk refresh`
- The alternative is incremental load
- Good for small datasets, dimension tables.

Ingestion 101: Reflections

✓ Providers and Hooks 201

✓ Wrote your own DAG

✓ Idempotency

✓ Full Load

Next Goal

WHAT IF THE PUBLISHER SENDS A NEW CATALOG EVERY WEEK?

Each file may contain old books with updated prices and new books that were not there before.

`books-01-05-2026.csv`

`books-07-05-2026.csv`

`books-14-05-2026.csv`

Next Goal

WHAT IF THE PUBLISHER SENDS A NEW CATALOG EVERY WEEK?

Each file may contain old books with updated prices and new books that were not there before.

books-01-05-2026.csv

books-07-05-2026.csv

books-14-05-2026.csv

ONE DAY, SOMEONE WIPES THE PROD DATABASE.

How do you recover? How do you ensure the same pipelines fetch the same data, in the right order, keeping the catalog up to date?

Next Goal

WHAT IF THE PUBLISHER SENDS A NEW CATALOG EVERY WEEK?

Each file may contain old books with updated prices and new books that were not there before.

books-01-05-2026.csv

books-07-05-2026.csv

books-14-05-2026.csv

ONE DAY, SOMEONE WIPES THE PROD DATABASE.

How do you recover? How do you ensure the same pipelines fetch the same data, in the right order, keeping the catalog up to date?

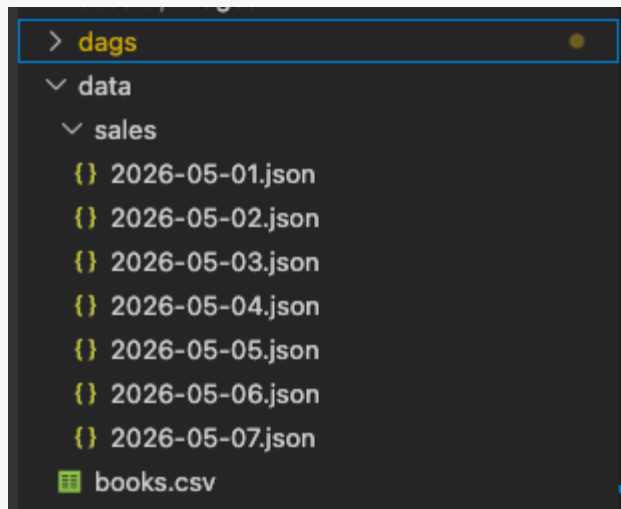
THAT IS OUR NEXT EXERCISE.

Module 2

Daily Sales Ingest

(Incremental)

Scheduling in Airflow



Scheduling in Airflow

```
@dag(  
    schedule="@daily",          # run once per day  
    start_date=datetime(2026, 5, 1),  
    catchup=True,              # backfill all days since start_date  
)
```

Common schedule values

Scheduling in Airflow

```
@dag(  
    schedule="@daily",          # run once per day  
    start_date=datetime(2026, 5, 1),  
    catchup=True,              # backfill all days since start_date  
)
```

Common schedule values

- `None` — manual trigger only

Scheduling in Airflow

```
@dag(  
    schedule="@daily",          # run once per day  
    start_date=datetime(2026, 5, 1),  
    catchup=True,              # backfill all days since start_date  
)
```

Common schedule values

- `None` — manual trigger only
- `"@daily"` — once per day at midnight

Scheduling in Airflow

```
@dag(  
    schedule="@daily",          # run once per day  
    start_date=datetime(2026, 5, 1),  
    catchup=True,              # backfill all days since start_date  
)
```

Common schedule values

- `None` — manual trigger only
- `"@daily"` — once per day at midnight
- `"@hourly"` — once per hour

Scheduling in Airflow

```
@dag(  
    schedule="@daily",          # run once per day  
    start_date=datetime(2026, 5, 1),  
    catchup=True,              # backfill all days since start_date  
)
```

Common schedule values

- `None` — manual trigger only
- `"@daily"` — once per day at midnight
- `"@hourly"` — once per hour
- `"0 6 * * *"` — cron: 6am every day

Catchup

```
@dag(  
    schedule="@daily",          # run once per day  
    start_date=datetime(2026, 5, 1),  
    catchup=True,              # backfill all days since start_date to today  
)
```

Catchup

```
@dag(  
    schedule="@daily",          # run once per day  
    start_date=datetime(2026, 5, 1),  
    catchup=True,              # backfill all days since start_date to today  
)
```

1. Start date = 1 May 2026
2. Dag written and activated on - 13th May 2026
3. Schedule = Daily
4. No of catchup DagRuns = 13

Catchup

```
@dag(  
    schedule="@daily",          # run once per day  
    start_date=datetime(2026, 5, 1),  
    catchup=True,              # backfill all days since start_date to today  
)
```

1. Start date = 1 May 2026
2. Dag written and activated on - 13th May 2026
3. Schedule = Daily
4. No of catchup DagRuns = 13

if it's scheduled weekly?

Catchup

```
@dag(  
    schedule="@daily",          # run once per day  
    start_date=datetime(2026, 5, 1),  
    catchup=True,               # backfill all days since start_date to today  
)
```

1. Start date = 1 May 2026
2. Dag written and activated on - 13th May 2026
3. Schedule = Daily
4. No of catchup DagRuns = 13

if it's scheduled weekly?

Catchup let's your DagRuns catchup(run) on all past schedules

Works only if `schedule ≠ None`

Logical Date

- The date the run *represents*, not when it actually ran.
- Eg., A dagrun scheduled at midnight might start running at `12:03`, but logical datetime will be `12:00`
- At a task level we can access them as parameters

```
@task
def load_file(ds=None):
    path = REPO_ROOT / "data" / "sales" / f"{ds}.json"
    # ^ "2026-05-03" for the May 3 run
    return json.loads(path.read_text())
```

- At Operator level we can access them as jinja templates

```
echo_date = BashOperator(
    task_id="echo_date",
    bash_command="echo {{ ds }}",
)
```

Mini-Exercise

Explore other template fields

<https://airflow.apache.org/docs/apache-airflow/stable/templates-ref.html>

XComs

XComs are an airflow construct to share small data between tasks.

Passing Values Between Tasks

```
@task
def insert_sales(ds=None):
    hook = PostgresHook(postgres_conn_id="bookshop_postgres")
    rows = ... # Assume we fetched the rows here
    return len(rows) # pushed to XCom automatically

@task
def log_summary(count, ds=None):
    print(f"Date: {ds} | Inserted: {count} records")
    # ^ pulled from XCom automatically & passed as argument

# Passing xcom while chaining
# denotes that the output `insert_sales` is input to `log_summary`
log_summary(insert_sales())
```

XComs

XComs are an airflow construct to share small data between tasks.

Passing Values Between Tasks

```
@task
def insert_sales(ds=None):
    hook = PostgresHook(postgres_conn_id="bookshop_postgres")
    rows = ... # Assume we fetched the rows here
    return len(rows) # pushed to XCom automatically

@task
def log_summary(count, ds=None):
    print(f"Date: {ds} | Inserted: {count} records")
    # ^ pulled from XCom automatically & passed as argument

# Passing xcom while chaining
# denotes that the output `insert_sales` is input to `log_summary`
log_summary(insert_sales())
```

- Return a value from `@task` and it is saved to XCom in the metadata DB

XComs

XComs are an airflow construct to share small data between tasks.

Passing Values Between Tasks

```
@task
def insert_sales(ds=None):
    hook = PostgresHook(postgres_conn_id="bookshop_postgres")
    rows = ... # Assume we fetched the rows here
    return len(rows) # pushed to XCom automatically

@task
def log_summary(count, ds=None):
    print(f>Date: {ds} | Inserted: {count} records")
    # ^ pulled from XCom automatically & passed as argument

# Passing xcom while chaining
# denotes that the output `insert_sales` is input to `log_summary`
log_summary(insert_sales())
```

- Return a value from `@task` and it is saved to XCom in the metadata DB
- Pass that return value as an argument to the next task — Airflow wires the pull automatically

XComs

XComs are an airflow construct to share small data between tasks.

Passing Values Between Tasks

```
@task
def insert_sales(ds=None):
    hook = PostgresHook(postgres_conn_id="bookshop_postgres")
    rows = ... # Assume we fetched the rows here
    return len(rows) # pushed to XCom automatically

@task
def log_summary(count, ds=None):
    print(f>Date: {ds} | Inserted: {count} records")
    # ^ pulled from XCom automatically & passed as argument

# Passing xcom while chaining
# denotes that the output `insert_sales` is input to `log_summary`
log_summary(insert_sales())
```

- Return a value from `@task` and it is saved to XCom in the metadata DB
- Pass that return value as an argument to the next task — Airflow wires the pull automatically

Exercise 2

Ingest Daily Sales

Load one JSON file per day using `ds`. Enable catchup and watch 7 backfill runs trigger.

```
dags/02_daily_sales_starter.py
```

Incremental Ingestion



Incremental Loading



Backfill Data



Scheduling Dags



Logical Date

Next Goal

WHAT IF WE WANT TO VALIDATE THE DATA AS AND WHEN WE COMPLETE INGESTION?

How will we schedule the downstream dag based on upstream's completion

Module 3

Validate and Approve

Assets

Assets

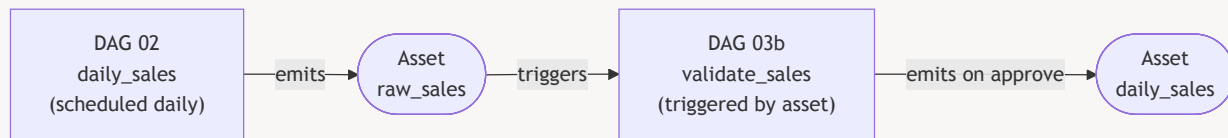
Problem: Time-based schedules don't know if the upstream DAG finished. A DAG scheduled at 6am might start before yesterday's data arrived.

Assets

Problem: Time-based schedules don't know if the upstream DAG finished. A DAG scheduled at 6am might start before yesterday's data arrived.

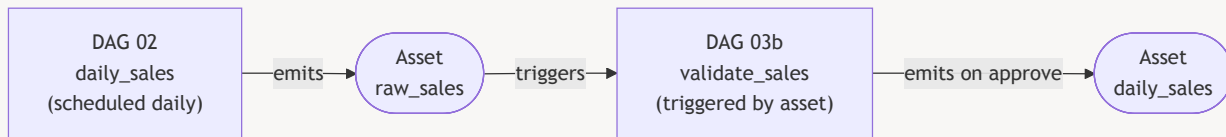
Solution: Assets – a DAG declares what data it produces; downstream DAGs run when that data is ready, not on a clock.

Pipeline With Asset



p

Pipeline With Asset



DAG 03b does not run on a clock. It wakes up the moment DAG 02 marks `raw_sales` ready.

p

Assets – Producer

```
@task(outlets=[Asset("raw_sales")])
def insert_sales(ds=None, **context):
    # ... insert rows ...

    # Attach metadata to the asset event
    context["outlet_events"][Asset("raw_sales")].extra = {
        "date": ds,
        "count": len(records),
    }
```

Assets – Producer

```
@task(outlets=[Asset("raw_sales")])
def insert_sales(ds=None, **context):
    # ... insert rows ...

    # Attach metadata to the asset event
    context["outlet_events"][Asset("raw_sales")].extra = {
        "date": ds,
        "count": len(records),
    }
```

- `outlets` declares what this task produces
- `outlet_events[asset].extra` attaches a dict that travels with the event

Assets – Consumer

```
@dag(schedule=Asset("raw_sales"))    # fires when raw_sales is updated
def downstream():
    @task
    def print_event(**context):
        events = context["triggering_asset_events"][Asset("raw_sales")]
        print(events[0].extra)
        # {"date": "2026-05-01", "count": 5}
```

Assets – Consumer

```
@dag(schedule=Asset("raw_sales")) # fires when raw_sales is updated
def downstream():
    @task
    def print_event(**context):
        events = context["triggering_asset_events"][Asset("raw_sales")]
        print(events[0].extra)
        # {"date": "2026-05-01", "count": 5}
```

- `schedule=Asset( ... )` replaces a cron string – no fixed time needed

Assets – Consumer

```
@dag(schedule=Asset("raw_sales")) # fires when raw_sales is updated
def downstream():
    @task
    def print_event(**context):
        events = context["triggering_asset_events"][Asset("raw_sales")]
        print(events[0].extra)
        # {"date": "2026-05-01", "count": 5}
```

- `schedule=Asset( ... )` replaces a cron string – no fixed time needed
- `triggering_asset_events` gives the consumer access to the producer's extra data

Exercise 3a

Asset Handoff

Data Quality

BAD DATA IS WORSE THAN NO DATA

A negative quantity silently loaded into `daily_sales` skews every report downstream.
By the time someone notices, the damage is done.

Data Quality

BAD DATA IS WORSE THAN NO DATA

A negative quantity silently loaded into `daily_sales` skews every report downstream. By the time someone notices, the damage is done.

WHAT OUR SALES FILES CONTAIN

Each daily JSON has 10 records. Two are bad: one has `quantity: -2` and one references an ISBN not in the books catalog.

Data Quality

BAD DATA IS WORSE THAN NO DATA

A negative quantity silently loaded into `daily_sales` skews every report downstream. By the time someone notices, the damage is done.

WHAT OUR SALES FILES CONTAIN

Each daily JSON has 10 records. Two are bad: one has `quantity: -2` and one references an ISBN not in the books catalog.

THE GOAL

Insert clean records. Quarantine bad ones with a reason. Then pause and ask a human -
- should we proceed?

Validate & quarantine

- One loop. Valid rows go to `daily_sales`. Bad rows go to `sales_quarantine`.
- If sale is negative or book is unknown quarantine the row

Human-in-the-Loop

```
from airflow.providers.standard.operators.hitl import ApprovalOperator

approve = ApprovalOperator(
    task_id="approve_or_reject",
    subject="Quarantined sales records require your review",
    body=""

    Date: {{ ds }}

    {{ ti.xcom_pull(task_ids='validate_and_insert') }}

    Approve to continue. Reject to stop this run.
    """
    outlets=[Asset("daily_sales")],
    response_timeout=timedelta(hours=24),
)
```

Human-in-the-Loop

```
from airflow.providers.standard.operators.hitl import ApprovalOperator

approve = ApprovalOperator(
    task_id="approve_or_reject",
    subject="Quarantined sales records require your review",
    body=""

    Date: {{ ds }}

    {{ ti.xcom_pull(task_ids='validate_and_insert') }}

    Approve to continue. Reject to stop this run.
    """
    outlets=[Asset("daily_sales")],
    response_timeout=timedelta(hours=24),
)
```

Airflow pauses the DAG and shows the quarantine table -- pulled from XCom -- inside an Approve/Reject form. No external tools needed.

Human-in-the-Loop

```
from airflow.providers.standard.operators.hitl import ApprovalOperator

approve = ApprovalOperator(
    task_id="approve_or_reject",
    subject="Quarantined sales records require your review",
    body=""

    Date: {{ ds }}

    {{ ti.xcom_pull(task_ids='validate_and_insert') }}

    Approve to continue. Reject to stop this run.
    """
    outlets=[Asset("daily_sales")],
    response_timeout=timedelta(hours=24),
)
```

Airflow pauses the DAG and shows the quarantine table -- pulled from XCom -- inside an Approve/Reject form. No external tools needed.

ON APPROVE

The `daily_sales` asset is emitted and DAG 04 triggers automatically.

Exercise 3b

Validate Sales + Human-in-the-Loop

Split valid and bad records, quarantine the bad ones, then approve in the Airflow UI.

```
dags/03b_validate_sales_starter.py
```

Data Validation

✓ Data Validation & Quarantining

✓ Assets

✓ HITL

Next Goal

```
known_isbns = {row[0] for row in hook.get_records("SELECT isbn FROM books")}

valid_rows, bad_rows = [], []
for rec in records:
    if rec["quantity"] <= 0:
        bad_rows.append( ... )
    elif rec["isbn"] not in known_isbns:
        bad_rows.append( ... )
    else:
        valid_rows.append( ... )
```

- What happens if the sales records grow in size? Instead of 10 sales, we get 10K sales.
- What if we want to parallelize the validation?
- Ideal way is to use SQL
- But is there an Airflow construct?

Module 4

Generate Report

Genre Report as a For-Loop

```
@task
def genre_report():
    genres = get_genres() # ["Fiction", "Mystery", "Science", ...]
    for genre in genres:
        build_genre_report(genre)
```

SEQUENTIAL BY DEFAULT

All genres run one after another inside a single task. The entire loop must finish before any result is visible in the UI.

Genre Report as a For-Loop

```
@task
def genre_report():
    genres = get_genres() # ["Fiction", "Mystery", "Science", ...]
    for genre in genres:
        build_genre_report(genre)
```

SEQUENTIAL BY DEFAULT

All genres run one after another inside a single task. The entire loop must finish before any result is visible in the UI.

ALL-OR-NOTHING RETRY

If one genre fails, the whole task fails. Every genre re-runs on retry — even the ones that already succeeded.

Genre Report as a For-Loop

```
@task
def genre_report():
    genres = get_genres() # ["Fiction", "Mystery", "Science", ...]
    for genre in genres:
        build_genre_report(genre)
```

SEQUENTIAL BY DEFAULT

All genres run one after another inside a single task. The entire loop must finish before any result is visible in the UI.

ALL-OR-NOTHING RETRY

If one genre fails, the whole task fails. Every genre re-runs on retry — even the ones that already succeeded.

ZERO VISIBILITY

Airflow shows one opaque task block. You cannot tell which genre is running, which finished, or which failed.

Dynamic Task Mapping

Dynamic Task Mapping

Step 1 - extract the loop body into its own `@task`

Dynamic Task Mapping

Step 1 - extract the loop body into its own `@task`

```
@task
def build_genre_report(genre):
    ...
```

Dynamic Task Mapping

Step 1 - extract the loop body into its own `@task`

```
@task
def build_genre_report(genre):
    ...
```

Step 2 - get the list

Dynamic Task Mapping

Step 1 - extract the loop body into its own `@task`

```
@task
def build_genre_report(genre):
    ...
```

Step 2 - get the list

```
@task
def get_genres():
    hook = PostgresHook(postgres_conn_id="bookshop_postgres")
    rows = hook.get_records("SELECT DISTINCT genre FROM books WHERE genre IS NOT
    NULL")
    return rows
```

Dynamic Task Mapping

Step 1 - extract the loop body into its own `@task`

```
@task
def build_genre_report(genre):
    ...
```

Step 2 - get the list

```
@task
def get_genres():
    hook = PostgresHook(postgres_conn_id="bookshop_postgres")
    rows = hook.get_records("SELECT DISTINCT genre FROM books WHERE genre IS NOT
    NULL")
    return rows
```

Step 3 - replace the loop with `.expand()`

Step 1 - extract the loop body into its own @task

Step 2 - get the list

Step 3 - replace the loop with `.expand()`

[illegible]

Dynamic Task Mapping

Step 1 - extract the loop body into its own `@task`

```
@task
def build_genre_report(genre):
    ...
```

Step 2 - get the list

```
@task
def get_genres():
    hook = PostgresHook(postgres_conn_id="bookshop_postgres")
    rows = hook.get_records("SELECT DISTINCT genre FROM books WHERE genre IS NOT
    NULL")
    return rows
```

Step 3 - replace the loop with `.expand()`

```
genre_rows = build_genre_report.expand(genre=genres)
#                                     ^ one task instance per genre
```

PARALLEL BY DEFAULT

Airflow creates one task instance per genre at runtime. They run in parallel, each with its own log and retry.

Exercise 4

Build the Genre Report

Aggregate sales per genre using `.expand()`, then watch DAG 04 trigger automatically when DAG 03 approves.

`dags/04_genre_report_starter.py`

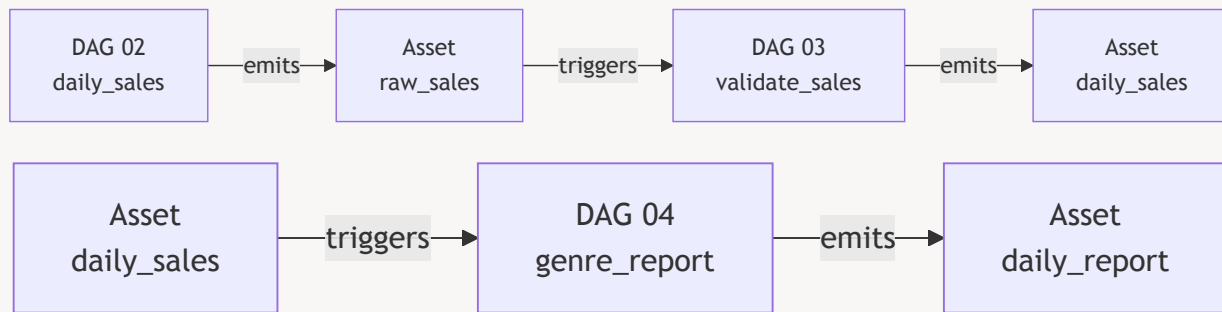
Generate Report

☒ Dynamic Task Mapping

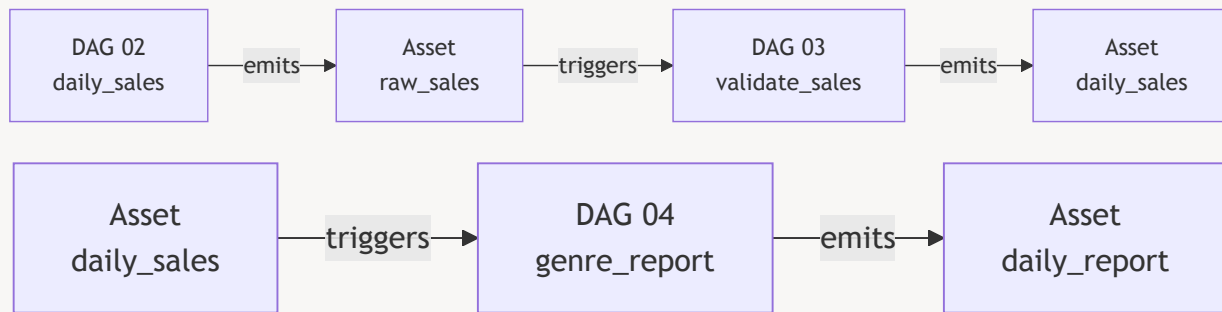
☒ Assets

☒ HITL

Final Pipeline

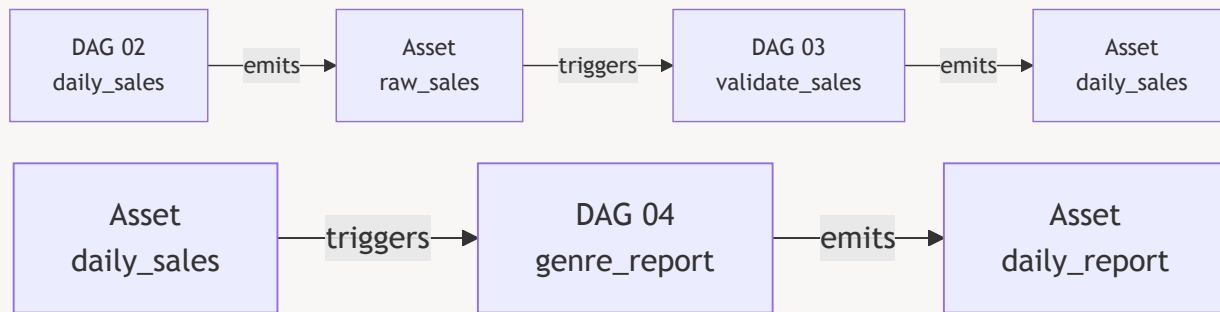


Final Pipeline



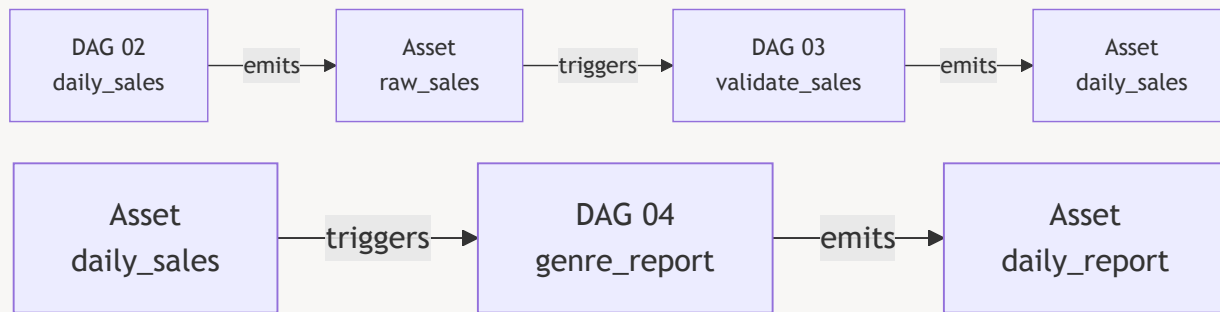
- Every downstream DAG wakes up on data, not on a clock

Final Pipeline



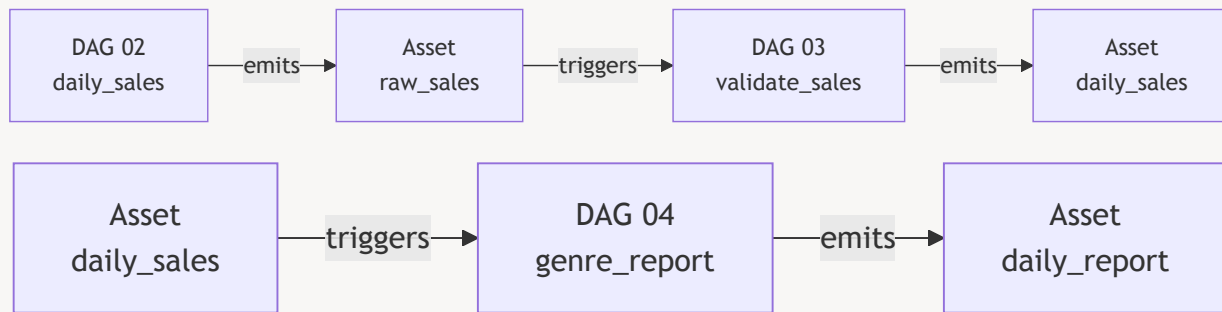
- Every downstream DAG wakes up on data, not on a clock
- Open Airflow UI **Assets** tab to see this graph live

Final Pipeline



- Every downstream DAG wakes up on data, not on a clock
- Open Airflow UI **Assets** tab to see this graph live
- If any DAG fails, nothing downstream runs on stale data

Final Pipeline



- Every downstream DAG wakes up on data, not on a clock
- Open Airflow UI **Assets** tab to see this graph live
- If any DAG fails, nothing downstream runs on stale data
- Each DAG can be maintained, retried, and tested independently

What You Learned

- DAG anatomy: `@dag`, `@task`, `>>`, manual trigger, Airflow UI
- Connections and PostgresHook — credentials out of code
- Scheduling, catchup, Jinja `{{ ds }}`, logical date, XCom
- Branching with `@task.branch`, trigger rules, quarantine pattern
- Dynamic Task Mapping with `.expand()` for parallel processing
- Assets - event-driven scheduling between decoupled DAGs
- Full-load, incremental data ingestion, validation, reporting

Explore Next

Go deeper on Airflow

Grow the system

Explore Next

Go deeper on Airflow

- **TaskGroups** — visually group related tasks inside a DAG
- **Sensors** — wait for a file, a table, or an external condition
- **KubernetesExecutor** — run each task in its own pod at scale
- **Deferrable operators** — async waiting without blocking a worker slot

Grow the system

Explore Next

Go deeper on Airflow

- **TaskGroups** — visually group related tasks inside a DAG
- **Sensors** — wait for a file, a table, or an external condition
- **KubernetesExecutor** — run each task in its own pod at scale
- **Deferrable operators** — async waiting without blocking a worker slot

Grow the system

- dbt for SQL transform ownership
- Great Expectations for richer data quality contracts
- Asset-aware orchestration with multiple upstream dependencies
- AI pipelines and operators

Thank You

Questions, ideas, and extensions are welcome.

<https://www.linkedin.com/in/bhavanicodes>
bhavanicodes@gmail.com

Appendix — Hosting Options

Option	Best for	Tradeoff
<code>airflow</code> <code>standalone</code>	Local dev, workshops	Single process, not for production
Docker Compose	Team dev, CI	Easy setup, executor limited to Local
Self-hosted Kubernetes	Full control, large scale	High ops overhead
Amazon MWAA	AWS-native shops	Managed, but version lag and limited config
Google Cloud Composer	GCP-native shops	Managed, expensive at small scale
Astronomer	Enterprise, any cloud	Fully managed + support, paid